POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

# SYLABUS PRZEDMIOTU

| | |
|---|---|
| **Nazwa przedmiotu:** | **Architektura mikroserwisowa i mikrofrontendowa** |
| **Kod przedmiotu:** | AMM |
| **Kierunek / Profil:** | Informatyka / praktyczny |
| **Tryb studiów:** | stacjonarny |
| **Rok / Semestr:** | 3 / 5 |
| **Charakter:** | obieralny |
| **Odpowiedzialny:** | mgr inż. Adam Urbanowicz |
| **Wersja z dnia:** | 26.02.2026 |

## 1. Godziny zajęć i punkty ECTS

| Wykłady | Ćwiczenia | Laboratori | Z prowadzącym | Praca własna | Łącznie | ECTS |
|---|---|---|---|---|---|---|
| 30 h | — | 30 h | 60 h | 65 h | 125 h | **5** |

## 2. Forma zajęć

| Forma zajęć | Sposób zaliczenia |
|---|---|
| Wykład | Egzamin |
| Laboratorium | Zaliczenie z oceną |

## 3. Cel dydaktyczny

Celem przedmiotu jest zapoznanie studentów z nowoczesnymi architekturami systemów rozproszonych, ze szczególnym uwzględnieniem wzorca mikroserwisów oraz mikrofrontendów. Student nabywa umiejętność projektowania, implementowania i wdrażania systemów złożonych z niezależnych, komunikujących się usług, a także budowania skalowalnych interfejsów użytkownika złożonych z autonomicznych modułów frontendowych. Przedmiot

kształci kompetencje niezbędne w środowisku przemysłowym, gdzie systemy monolityczne są stopniowo zastępowane przez elastyczne architektury chmurowe.

## 4. Przedmioty wprowadzające

| Przedmiot | Wymagane zagadnienia |
|---|---|
| PAI – Programowanie aplikacji internetowych | Umiejętność tworzenia aplikacji webowych z użyciem Node.js/JavaScript oraz podstawowa znajomość protokołu HTTP i REST API. |
| SOP – Systemy operacyjne | Podstawowa znajomość systemów operacyjnych Linux, obsługi terminala, procesów i zarządzania siecią. |
| TIN – Technologie internetu | Znajomość HTML, CSS, JavaScript po stronie klienta. |

## 5. Treści programowe

1. W01. Architektura monolityczna vs. mikroserwisowa – ewolucja podejść do budowy systemów. Definicja mikroserwisu. Zalety i wady. Kiedy stosować mikroserwisy, a kiedy monolit jest lepszym wyborem.
2. W02. Domain-Driven Design jako podstawa dekompozycji. Bounded Context, ubiquitous language, agregaty, encje, obiekty wartości. Identyfikowanie granic mikroserwisów.
3. W03. Komunikacja synchroniczna: REST i gRPC. Projektowanie kontraktów API (OpenAPI/Swagger). Wersjonowanie API. Zasady idempotentności.
4. W04. Komunikacja asynchroniczna: message brokery (RabbitMQ, Apache Kafka). Wzorce: Publish/Subscribe, Event-Driven Architecture. Porównanie brokerów.
5. W05. Wzorzec API Gateway: routing, load balancing, autentykacja, rate limiting. Service Discovery: podejście client-side i server-side (Consul, Eureka).
6. W06. Odporność na błędy: wzorzec Circuit Breaker (Resilience4j, Hystrix), retry, timeout, bulkhead. Distributed tracing (Jaeger, Zipkin). Centralne logowanie (ELK Stack).
7. W07. Zarządzanie danymi w mikroserwisach: database-per-service, eventual consistency, wzorce Saga (choreography vs. orchestration) i CQRS.
8. W08. Event Sourcing: przechowywanie stanu jako sekwencji zdarzeń. Zalety i ograniczenia. Integracja z CQRS.
9. W09. Mikrofrontendy: motywacja, definicja, podejścia integracji (build-time, runtime, iframe, Web Components, Module Federation). Porównanie podejść.
10. W10. Webpack 5 Module Federation w szczegółach: host, remote, shared dependencies, dynamic imports. Wzorzec Shell Application.
11. W11. Routing i zarządzanie stanem w aplikacjach mikrofrontendowych. Komunikacja między modułami: Custom Events, shared state, Event Bus.

12. W12. Konteneryzacja: Docker w ekosystemie mikroserwisów. Dockerfile best practices, multi-stage build, sieci Docker, volumes. Docker Compose dla środowisk developerskich.
13. W13. Kubernetes: architektura klastra (control plane, worker node). Obiekty: Pod, Deployment, Service, ConfigMap, Secret, Ingress. Skalowanie horyzontalne (HPA).
14. W14. Bezpieczeństwo w architekturze mikroserwisowej: OAuth 2.0 i JWT na poziomie API Gateway, mTLS dla komunikacji wewnętrznej, zarządzanie sekretami (Vault).
15. W15. Wzorce organizacyjne: Conway's Law, Team Topologies (stream-aligned, enabling, complicated subsystem, platform teams). Strategie migracji z monolitu: Strangler Fig Pattern. Podsumowanie kursu.
16. L01. Konfiguracja środowiska: Docker Desktop, Node.js, VS Code. Uruchomienie pierwszego mikroserwisu w Node.js/Express. Dockeryzacja. Testowanie endpointów (Postman/curl).
17. L02. Projektowanie systemu – dekompozycja domeny wybranej aplikacji (np. sklep internetowy) na mikroserwisy metodą Event Storming. Definiowanie kontraktów REST (OpenAPI).
18. L03. Implementacja mikroserwisu nr 1 (np. serwis produktów) – REST API, walidacja, obsługa błędów, testy jednostkowe (Jest).
19. L04. Implementacja mikroserwisu nr 2 (np. serwis zamówień) – komunikacja synchroniczna (REST) z serwisem produktów. Obsługa scenariuszy błędów.
20. L05. Konfiguracja RabbitMQ w Docker Compose. Implementacja komunikacji asynchronicznej między serwisami (zdarzenie OrderPlaced → serwis notyfikacji).
21. L06. Implementacja API Gateway w NGINX lub jako mikroserwis Node.js – routing do serwisów backendu, uwierzytelnianie JWT, rate limiting.
22. L07. Wzorzec Circuit Breaker z użyciem biblioteki opossum (Node.js). Symulowanie awarii serwisu i testowanie mechanizmów fallback.
23. L08. Podstawy Webpack Module Federation – konfiguracja hosta i remotu. Pierwszy mikrofrontend wystawiający komponent React/Vue.
24. L09. Budowa aplikacji Shell – integracja co najmniej dwóch niezależnych modułów mikrofrontendowych. Dynamiczne ładowanie modułów.
25. L10. Routing w aplikacji mikrofrontendowej. Komunikacja między modułami przy użyciu Custom Events i współdzielonego Event Bus.
26. L11. Przygotowanie pełnego środowiska Docker Compose: backend (mikroserwisy + broker + baza danych) + frontend (shell + moduły). Demonstracja pełnej aplikacji.
27. L12. Wdrożenie na Kubernetes (lokalnie: Minikube lub k3s) – tworzenie manifestów Deployment i Service dla każdego mikroserwisu. Konfiguracja Ingress.
28. L13. Skalowanie i monitorowanie w Kubernetes – HPA, metryki (kubectl top), probes (liveness, readiness). Podstawy Helm: pakietowanie aplikacji jako Helm chart.
29. L14. Przegląd i obrona projektów końcowych – prezentacja architektury, demonstracja działającego systemu, dyskusja decyzji projektowych.
30. L15. Podsumowanie zajęć laboratoryjnych, omówienie wyników projektów, wskazówki do dalszego rozwoju (cloud providers: AWS ECS/EKS, GCP GKE, Azure AKS).

## 6. Przedmiotowe efekty uczenia się

## Wiedza

- Student zna i rozumie różnice między architekturą monolityczną a mikroserwisową: zalety, wady, kryteria wyboru oraz typowe scenariusze migracji.
- Student zna i rozumie zasady dekompozycji systemu na mikroserwisy z użyciem Domain-Driven Design (DDD), bounded context i reguł spójności domenowej.
- Student zna i rozumie mechanizmy komunikacji między mikroserwisami: wywołania synchroniczne (REST, gRPC) oraz asynchroniczne (message broker: RabbitMQ, Kafka) i potrafi wskazać zastosowania każdego z podejść.
- Student zna i rozumie wzorce architektoniczne stosowane w mikroserwisach: API Gateway, Service Discovery, Circuit Breaker, Saga, CQRS, Event Sourcing.
- Student zna i rozumie koncepcję mikrofrontendów: podział aplikacji frontendowej na niezależne moduły wdrażane i rozwijane przez oddzielne zespoły, w tym podejście Module Federation (Webpack 5).
- Student zna i rozumie podstawy konteneryzacji (Docker) i orkiestracji (Kubernetes) w kontekście wdrażania i skalowania mikroserwisów.
- Student zna i rozumie zagadnienia bezpieczeństwa w architekturze mikroserwisowej: uwierzytelnianie i autoryzacja (OAuth 2.0/JWT) na poziomie API Gateway, szyfrowanie komunikacji wewnętrznej.

## Umiejętności

- Student potrafi zaprojektować architekturę systemu opartego na mikroserwisach, dokonując dekompozycji domeny biznesowej, identyfikując granice serwisów i definiując kontrakty API.
- Student potrafi zaimplementować co najmniej trzy współpracujące mikroserwisy w technologii Node.js/Express lub Spring Boot, stosując komunikację synchroniczną (REST) i asynchroniczną (message broker).
- Student potrafi skonteneryzować mikroserwisy za pomocą Dockera, tworzyć pliki Dockerfile i konfiguracje Docker Compose do uruchamiania całego systemu lokalnie.
- Student potrafi zaprojektować i zaimplementować aplikację mikrofrontendową z użyciem Webpack Module Federation, integrując co najmniej dwa niezależne moduły frontendowe w ramach wspólnego shell-a.
- Student potrafi skonfigurować i wdrożyć API Gateway (np. NGINX, Kong lub własny w Node.js) obsługujące routing, load balancing i uwierzytelnianie.
- Student potrafi zidentyfikować wąskie gardła i punkty awarii w architekturze mikroserwisowej oraz zastosować wzorzec Circuit Breaker i mechanizmy retry/fallback.
- Student potrafi przygotować podstawową konfigurację klastra Kubernetes (Deployment, Service, Ingress) i wdrożyć na nim skonteneryzowany system mikroserwisowy.

## Kompetencje społeczne

- Student jest gotów do pracy w modelu organizacyjnym opartym na autonomicznych zespołach (tzw. team topologies), rozumiejąc wpływ struktury zespołu na architekturę systemu (prawo Conwaya).

- Student jest gotów do krytycznej oceny zasadności stosowania architektury mikroserwisowej w danym projekcie, uwzględniając koszty złożoności operacyjnej i organizacyjnej.
- Student jest gotów do samodzielnego śledzenia aktualnych trendów w inżynierii oprogramowania i infrastrukturze chmurowej oraz aktualizowania swojej wiedzy w oparciu o dokumentację techniczną i społeczność open source.

## 7. Kryteria oceny

- Wykład – egzamin pisemny:
- Egzamin pisemny zawierający pytania otwarte i zamknięte sprawdzające wiedzę teoretyczną z zakresu architektury mikroserwisowej i mikrofrontendowej. Warunkiem zaliczenia jest uzyskanie co najmniej 51% punktów.
- Skala ocen: 51–60% – 3.0, 61–70% – 3.5, 71–80% – 4.0, 81–90% – 4.5, 91–100% – 5.0.
- Laboratorium – zaliczenie z oceną:
- Ocena ciągła na podstawie sprawozdań z zajęć laboratoryjnych (40% oceny końcowej laboratorium).
- Projekt końcowy: zaprojektowanie i zaimplementowanie systemu złożonego z co najmniej 3 mikroserwisów komunikujących się przez REST/message broker oraz odpowiadającego mikrofrontendu opartego na Module Federation (60% oceny końcowej laboratorium).
- Warunkiem zaliczenia laboratorium jest oddanie wszystkich sprawozdań oraz obrona projektu końcowego.
- Ocena końcowa przedmiotu: średnia ważona oceny z egzaminu (60%) i oceny z laboratorium (40%).

## 8. Metody dydaktyczne

**wyklad:**

- Wykład z prezentacją multimedialną, demonstracje na żywo, analiza przypadków przemysłowych (Netflix, Amazon, Spotify).

**laboratorium:**

- Ćwiczenia praktyczne: projektowanie i implementacja mikroserwisów oraz mikrofrontendów z wykorzystaniem narzędzi branżowych (Docker, Kubernetes, Module Federation, Node.js).

## 9. Literatura

**Podstawowa:**

- S. Newman: "Budowanie mikroserwisów. Projektowanie drobnoziarnistych systemów", Helion, Gliwice 2022 (Building Microservices, 2nd ed., O'Reilly 2021).

- C. Richardson: "Mikrousługi. Wzorce projektowe i wdrażanie aplikacji", Helion, Gliwice 2020 (Microservices Patterns, Manning 2018).
- L. Jackson: "Mikrofrontendy w akcji", tłum. własne na podstawie "Micro Frontends in Action", Manning 2020.
- B. Burns, J. Beda, K. Hightower: "Kubernetes. Tworzenie niezawodnych systemów rozproszonych", Helion, Gliwice 2023.

**Uzupełniająca:**

- M. Fowler: artykuły na martinfowler.com dot. mikroserwisów, event sourcingu i CQRS.
- dokumentacja oficjalna: Webpack Module Federation (webpack.js.org), Docker (docs.docker.com), Kubernetes (kubernetes.io/docs).
- M. Kleppmann: "Przetwarzanie danych w dużej skali. Niezawodność, skalowalność i łatwość konserwacji systemów", Helion 2021.